

Software Requirement Specification (SRS)

SkillSync - AI-Powered Project Management System

1. Introduction

1.1 Purpose

The purpose of this system is to build an **AI-Powered Project Management Platform** with intelligent skill-based team assembly, automated task assignment, predictive analytics, and real-time collaboration features. The application will serve as an intelligent workspace where organizations can optimize project staffing, track capacity, and improve project success rates through data-driven insights.

1.2 Scope

The platform will provide:

- **Smart Team Assembly:** AI automatically suggests optimal team compositions based on required skills, availability, and past collaboration success rates.
- **Skill-Based Project Matching:** Projects are matched to team members based on skill proficiency, availability, and workload.
- **Predictive Analytics:** AI predicts project success probability, identifies potential bottlenecks, and suggests interventions.
- **Dynamic Task Assignment:** Tasks are automatically assigned based on skills, current workload, and capacity.
- **Real-Time Collaboration:** Team communication through comments, mentions, notifications, and activity feeds.
- **Capacity Management:** Visual dashboards showing team availability, workload distribution, and resource utilization.
- **Skill Gap Analysis:** Identifies missing skills for projects and recommends learning resources.
- **Project Brief Parser:** AI extracts requirements, skills, and timelines from uploaded project documents.

2. Technologies

- **Frontend:** React, React Router, Zustand (state management), Tailwind CSS, shadcn/ui (components), React Flow (visual graphs), Recharts (analytics)

- **Backend:** Node.js with Express (or Python with FastAPI), PostgreSQL, Redis (caching)
- **AI/ML:** OpenAI GPT-4 API, LangChain (AI workflows), Sentence Transformers (skill matching), scikit-learn (predictive models)
- **Real-Time:** Socket.io (notifications, live updates)
- **Authentication:** JWT-based authentication with bcrypt password hashing
- **File Storage:** AWS S3 or Cloudinary (project briefs, attachments)
- **Deployment:** Docker, Docker Compose, CI/CD pipeline

3. User Roles and Permissions

3.1 Team Member (Standard User)

- View assigned projects and tasks
- Update task status and log time
- Upload files and add comments to tasks
- View own skill profile and development recommendations
- Access available projects matched to their skills
- Track personal workload and capacity
- Participate in team discussions and mentions
- View project dashboards and analytics

3.2 Project Manager

- Create and configure new projects
- Use AI to assemble teams based on skill requirements
- Assign and reassign tasks to team members
- View team capacity and workload distribution
- Access project success probability scores
- Upload project briefs for AI parsing
- Monitor project health and receive AI alerts
- Approve or modify AI-suggested team compositions
- Access advanced analytics and reporting
- Manage project timelines and milestones

3.3 Admin (System Administrator)

- Manage all users (create, edit, deactivate accounts)
- Assign roles and permissions (Member, Project Manager, Admin)
- View organization-wide analytics and metrics
- Access admin dashboard with user statistics
- Manage skill taxonomy and categories
- Configure AI model parameters and thresholds
- View audit logs and system activities
- Manage organizational settings and preferences

- Export data and generate reports
- Add or remove other administrators

4. Functional Requirements

4.1 Authentication Module

- Users must register with email and password
- Password hashing using bcrypt
- JWT-based session management
- Role assignment during registration (default: Team Member)
- Password reset via email verification
- Unauthorized users cannot access protected routes
- Session timeout after 24 hours of inactivity

4.2 User Profile Module

User profiles include:

- Personal Information: Name, email, role, bio, profile picture
- Skills: Array of skills with proficiency levels (Beginner/Intermediate/Advanced/Expert)
- Availability: Working hours per week, current capacity percentage
- Project History: List of completed and ongoing projects
- Performance Metrics: On-time delivery rate, collaboration score
- Learning Recommendations: AI-suggested courses for skill gaps
- Activity Feed: Recent activities and contributions

4.3 Project Management Module

Projects contain:

- Project Name, Description, Status (Planning/Active/Completed/On Hold)
- Required Skills: Array of skills with required proficiency levels
- Timeline: Start date, end date, milestones
- Team Members: Assigned users with their roles in the project
- Tasks: List of tasks with assignments and dependencies
- AI Success Score: Predicted probability of on-time completion (0-100%)
- Budget and Resources (optional)
- Attachments: Project briefs, documents, images

Project Views:

- Kanban Board (drag-and-drop task cards)
- List View (table format)
- Calendar View (tasks on timeline)

- Gantt Chart (optional, for timeline visualization)

4.4 AI-Powered Features

4.4.1 Smart Team Assembly

- Input: Project requirements (required skills, timeline, complexity)
- AI analyzes available team members and their skills
- Calculates skill coverage percentage for each potential team
- Considers past collaboration success rates between members
- Balances workload across the organization
- Outputs: Top 3 recommended team compositions with justifications
- Project Manager can accept, modify, or reject suggestions

4.4.2 Skill-Based Project Matching

- AI calculates match score (0-100%) between user skills and project requirements
- Uses cosine similarity on skill vectors for semantic matching
- Considers skill proficiency levels and recency
- Dashboard shows "Available Projects" ranked by match score
- Users can express interest in projects
- Notifications sent when high-match projects become available

4.4.3 Dynamic Task Auto-Assignment

- When new tasks are created, AI suggests optimal assignees
- Factors: Required skills, current workload, availability, past performance
- Can be configured as automatic or manual approval
- Re-balancing triggered when team member becomes overloaded
- Task redistribution suggestions when deadlines are at risk

4.4.4 Predictive Project Analytics

- Success Probability Score: Predicts on-time completion likelihood
- Based on: Team composition, historical data, task complexity, timeline
- Risk Detection: Identifies potential bottlenecks before they occur
- Alerts sent when success probability drops below threshold (60%)
- Resource Optimization: Suggests team adjustments or timeline changes
- Learning: Model improves accuracy over time with more project data

4.4.5 Project Brief Parser

- Upload PDF/DOCX project briefs
- AI (GPT-4) extracts: Required skills, timeline, budget, deliverables
- Auto-generates project structure with milestones
- Suggests team size based on scope
- Pre-fills project creation form
- User reviews and confirms or edits extracted information

4.4.6 Skill Gap Detection & Learning Recommendations

- Compares team skills against project requirements
- Identifies missing or under-proficient skills
- Recommends specific courses (Udemy, Coursera, LinkedIn Learning)
- Prioritizes gaps based on project importance and urgency
- Tracks skill development over time
- Updates user skills automatically based on completed projects

4.5 Task Management Module

- Create tasks with: Title, description, assigned user, due date, priority
- Task Status: To Do → In Progress → In Review → Done
- Subtasks support for breaking down complex work
- Task Dependencies: Specify which tasks must be completed first
- Time Tracking: Start/stop timer, manual time entries
- Comments: Thread-based discussions on tasks
- @Mentions: Tag team members for attention
- File Attachments: Upload relevant documents or images
- Task History: Audit trail of all changes

4.6 Collaboration Module

- Real-Time Notifications: Task assignments, mentions, comments, due dates
- Activity Feed: Project-level and personal activity streams
- In-App Messaging: Direct messages and group discussions
- Notification Preferences: Configure email vs in-app notifications
- Mention System: @username to notify specific team members
- Notification History: View all past notifications

4.7 Dashboard & Reporting Module

Personal Dashboard:

- My Tasks (filtered by status, priority)
- Available Projects (skill-matched)
- Current Workload (hours this week)
- Skill Development Recommendations
- Recent Activity
- Upcoming Deadlines

Project Dashboard:

- Project Progress (% complete)
- Tasks by Status (pie chart)
- Team Workload Distribution (bar chart)

- AI Success Score with trend
- Upcoming Milestones
- Risk Alerts
- Recent Activity

Admin Dashboard:

- Total Users, Projects, Tasks
- User Profiles with Skill Matrix
- Organizational Capacity Overview
- Project Success Rates
- Resource Utilization Metrics
- Skill Gap Analysis (organization-wide)
- User Activity Logs

4.8 Search & Filtering Module

- Global Search: Across projects, tasks, users, files
- Advanced Filters: By status, priority, assignee, due date, tags
- Skill-Based Search: Find users with specific skills
- Project Search: By name, status, team members
- Saved Filters: Create and save custom filter combinations

5. Non-Functional Requirements

Performance: API responses under 2 seconds, Dashboard loads under 3 seconds, Real-time notifications delivered within 1 second

Scalability: Support 1000+ concurrent users, Handle 10,000+ projects, Horizontal scaling with load balancers

Security: JWT authentication with token expiration, Password hashing with bcrypt (10 rounds), Role-based access control (RBAC), Input validation and sanitization, SQL injection prevention, XSS protection

Reliability: Database replication for data persistence, Automated backups every 24 hours, 99.5% uptime target, Error logging and monitoring

Usability: Responsive design (mobile, tablet, desktop), Intuitive UI with consistent design patterns, Accessibility compliance (WCAG 2.1), Loading indicators for all async operations

Maintainability: Modular codebase with clear separation of concerns, Comprehensive API documentation, Unit tests for core functions, Integration tests for API endpoints

6. Database Models (PostgreSQL Schema)

6.1 Users Table

- id (UUID, Primary Key)
- email (String, Unique, Not Null)
- password_hash (String, Not Null)
- name (String, Not Null)
- role (Enum: MEMBER, PROJECT_MANAGER, ADMIN)
- bio (Text, Optional)
- profile_picture_url (String, Optional)
- availability_hours_per_week (Integer, Default: 40)
- current_capacity_percentage (Integer, 0-100)
- created_at (Timestamp)
- updated_at (Timestamp)

6.2 Skills Table

- id (UUID, Primary Key)
- name (String, Unique, Not Null) - e.g., "React", "Python", "Project Management"
- category (String) - e.g., "Frontend", "Backend", "Soft Skills"
- created_at (Timestamp)

6.3 UserSkills Table (Junction Table)

- id (UUID, Primary Key)
- user_id (UUID, Foreign Key → Users)
- skill_id (UUID, Foreign Key → Skills)
- proficiency_level (Enum: BEGINNER, INTERMEDIATE, ADVANCED, EXPERT)
- verified (Boolean, Default: False) - admin can verify skills
- last_used (Timestamp)
- created_at (Timestamp)

6.4 Projects Table

- id (UUID, Primary Key)
- name (String, Not Null)
- description (Text)
- status (Enum: PLANNING, ACTIVE, COMPLETED, ON_HOLD)
- start_date (Date)
- end_date (Date)
- budget (Decimal, Optional)
- created_by (UUID, Foreign Key → Users)
- ai_success_score (Integer, 0-100, Default: NULL)
- created_at (Timestamp)
- updated_at (Timestamp)

6.5 ProjectSkills Table (Required Skills for Projects)

- id (UUID, Primary Key)
- project_id (UUID, Foreign Key → Projects)
- skill_id (UUID, Foreign Key → Skills)
- required_proficiency (Enum: BEGINNER, INTERMEDIATE, ADVANCED, EXPERT)
- is_critical (Boolean, Default: False)
- created_at (Timestamp)

6.6 ProjectMembers Table (Team Assignments)

- id (UUID, Primary Key)
- project_id (UUID, Foreign Key → Projects)
- user_id (UUID, Foreign Key → Users)
- role_in_project (String) - e.g., "Frontend Developer", "Team Lead"
- assigned_at (Timestamp)

6.7 Tasks Table

- id (UUID, Primary Key)
- project_id (UUID, Foreign Key → Projects)
- title (String, Not Null)
- description (Text)
- assigned_to (UUID, Foreign Key → Users, Optional)
- status (Enum: TODO, IN_PROGRESS, IN_REVIEW, DONE)
- priority (Enum: LOW, MEDIUM, HIGH, URGENT)
- due_date (Timestamp, Optional)
- estimated_hours (Decimal, Optional)
- actual_hours (Decimal, Default: 0)
- parent_task_id (UUID, Foreign Key → Tasks, Optional) - for subtasks
- created_by (UUID, Foreign Key → Users)
- created_at (Timestamp)
- updated_at (Timestamp)

6.8 TaskDependencies Table

- id (UUID, Primary Key)
- task_id (UUID, Foreign Key → Tasks) - the dependent task
- depends_on_task_id (UUID, Foreign Key → Tasks) - the task it depends on
- created_at (Timestamp)

6.9 Comments Table

- id (UUID, Primary Key)
- task_id (UUID, Foreign Key → Tasks)
- user_id (UUID, Foreign Key → Users)
- content (Text, Not Null)

- created_at (Timestamp)
- updated_at (Timestamp)

6.10 Notifications Table

- id (UUID, Primary Key)
- user_id (UUID, Foreign Key → Users)
- type (Enum: TASK_ASSIGNED, MENTION, COMMENT, DEADLINE, ALERT)
- title (String)
- message (Text)
- link (String) - URL to relevant resource
- is_read (Boolean, Default: False)
- created_at (Timestamp)

6.11 SkillMatchScores Table (Cached Match Results)

- id (UUID, Primary Key)
- user_id (UUID, Foreign Key → Users)
- project_id (UUID, Foreign Key → Projects)
- match_score (Integer, 0-100)
- skill_coverage_percentage (Integer, 0-100)
- availability_fit (Integer, 0-100)
- workload_fit (Integer, 0-100)
- calculated_at (Timestamp)

6.12 LearningRecommendations Table

- id (UUID, Primary Key)
- user_id (UUID, Foreign Key → Users)
- skill_id (UUID, Foreign Key → Skills)
- current_level (Enum: NONE, BEGINNER, INTERMEDIATE, ADVANCED)
- target_level (Enum: BEGINNER, INTERMEDIATE, ADVANCED, EXPERT)
- reason (Text) - why this skill is recommended
- course_url (String, Optional)
- course_name (String, Optional)
- priority (Enum: LOW, MEDIUM, HIGH)
- created_at (Timestamp)

7. Key API Endpoints

7.1 Authentication APIs

- POST /api/auth/register - User registration
- POST /api/auth/login - User login (returns JWT)
- POST /api/auth/logout - Logout
- POST /api/auth/forgot-password - Password reset request

- POST /api/auth/reset-password - Reset password with token

7.2 Users APIs

- GET /api/users/me - Get current user profile
- PUT /api/users/me - Update current user profile
- GET /api/users/:id - Get user by ID
- GET /api/users - List all users (admin only)
- PUT /api/users/:id/role - Update user role (admin only)

7.3 Skills APIs

- GET /api/skills - List all skills
- POST /api/skills - Create new skill (admin)
- POST /api/users/me/skills - Add skill to current user
- PUT /api/users/me/skills/:id - Update skill proficiency
- DELETE /api/users/me/skills/:id - Remove skill

7.4 Projects APIs

- GET /api/projects - List all projects
- GET /api/projects/:id - Get project details
- POST /api/projects - Create new project
- PUT /api/projects/:id - Update project
- DELETE /api/projects/:id - Delete project
- GET /api/projects/available - Get skill-matched available projects
- POST /api/projects/:id/team - Add team member
- DELETE /api/projects/:id/team/:userId - Remove team member

7.5 AI Features APIs

- POST /api/ai/team-assembly - Get AI team suggestions
- POST /api/ai/parse-brief - Parse uploaded project brief
- GET /api/ai/project-match/:projectId - Get user match scores
- POST /api/ai/predict-success/:projectId - Calculate success probability
- GET /api/ai/skill-gaps/:projectId - Identify skill gaps
- GET /api/ai/learning-recommendations - Get personalized learning paths

7.6 Tasks APIs

- GET /api/tasks - List tasks (with filters)
- GET /api/tasks/:id - Get task details
- POST /api/tasks - Create new task
- PUT /api/tasks/:id - Update task
- DELETE /api/tasks/:id - Delete task
- POST /api/tasks/:id/time - Log time on task
- POST /api/tasks/:id/comments - Add comment to task

7.7 Notifications APIs

- GET /api/notifications - Get user notifications
- PUT /api/notifications/:id/read - Mark as read
- PUT /api/notifications/read-all - Mark all as read
- DELETE /api/notifications/:id - Delete notification

7.8 Dashboard APIs

- GET /api/dashboard/personal - Personal dashboard data
- GET /api/dashboard/project/:id - Project dashboard data
- GET /api/dashboard/admin - Admin dashboard (admin only)

8. System Flow

8.1 User Onboarding

1. User registers with email and password
2. System creates user account with default role (Team Member)
3. User completes profile (adds skills, availability)
4. System calculates initial capacity and match scores
5. User redirected to personal dashboard

8.2 Project Creation & Team Assembly

1. Project Manager creates new project
2. PM enters project details and required skills
3. PM optionally uploads project brief for AI parsing
4. AI analyzes requirements and suggests team compositions
5. PM reviews suggestions and selects team
6. System sends notifications to assigned team members
7. AI calculates initial success probability
8. Project appears on team members' dashboards

8.3 Task Assignment & Execution

1. PM or system creates tasks within project
2. AI suggests optimal assignees based on skills and workload
3. Tasks assigned (manual or automatic)
4. Team members receive notifications
5. Members update task status as work progresses
6. Comments and mentions used for collaboration
7. Time tracked automatically or manually
8. Task completion updates project progress

8.4 AI Monitoring & Alerts

1. AI continuously monitors project metrics

- 2. Detects risks: low success score, overloaded members, missed deadlines
- 3. Sends alerts to Project Manager
- 4. Suggests interventions: rebalancing, timeline adjustment
- 5. PM takes action or dismisses alert
- 6. System logs decision for learning

8.5 Skill Development Cycle

- 1. AI identifies skill gaps for active projects
- 2. Generates learning recommendations for users
- 3. User views recommendations in dashboard
- 4. User completes courses (external)
- 5. User or admin updates skill proficiency
- 6. System recalculates match scores
- 7. User becomes eligible for new projects

9. AI Algorithms & Models

9.1 Skill Matching Algorithm

Uses cosine similarity on skill vectors:

- Convert user skills to vector: [skill1_proficiency, skill2_proficiency, ...]
- Convert project requirements to vector: [required_proficiency1, ...]
- Calculate cosine similarity: $(A \cdot B) / (||A|| \times ||B||)$
- Normalize to 0-100 scale
- Weight by skill criticality and recency

9.2 Team Assembly Optimization

- Input: Required skills, project complexity, timeline
- Generate candidate teams (combinations of available users)
- Score each team on: Skill coverage, workload balance, past collaboration
- Use greedy algorithm to find top 3 teams
- Return with justifications and alternative suggestions

9.3 Success Prediction Model

- Features: Team size, skill coverage %, avg experience, timeline, complexity
- Historical data: Past projects with success/failure outcomes
- Model: Random Forest Classifier (scikit-learn)
- Outputs: Success probability (0-100%), risk factors
- Retrains monthly with new project data

9.4 Project Brief Parser

- Uses OpenAI GPT-4 API with structured output

- Prompt: "Extract required skills, timeline, budget, deliverables from this brief"
- Response format: JSON with predefined schema
- Validates extracted data against database constraints
- Confidence scoring for each extracted field

10. Security Considerations

- Password Requirements: Minimum 8 characters, at least 1 uppercase, 1 lowercase, 1 number
- JWT Tokens: 24-hour expiration, stored in httpOnly cookies
- Rate Limiting: 100 requests per minute per IP
- Input Validation: Server-side validation on all inputs
- SQL Injection Prevention: Parameterized queries only
- XSS Prevention: Content Security Policy headers, output encoding
- CORS: Whitelist approved frontend domains only
- File Uploads: Size limits (10MB), type restrictions, virus scanning
- Audit Logging: Log all sensitive operations (role changes, deletions)
- HTTPS Only: Enforce encrypted connections in production

11. Testing Strategy

Unit Tests: Test individual functions: skill matching, score calculations, validators - Target: 80% code coverage

Integration Tests: Test API endpoints: authentication, CRUD operations, AI features - Use Jest + Supertest

E2E Tests: Test user flows: registration, project creation, task assignment - Use Cypress or Playwright

Performance Tests: Load testing with 1000+ concurrent users - Use Apache JMeter or k6

Security Tests: Penetration testing, vulnerability scanning - Use OWASP ZAP

AI Model Tests: Validate prediction accuracy, test edge cases - Compare with baseline models

12. Deployment Architecture

- Frontend: React SPA deployed on Vercel or Netlify
- Backend: Node.js/Express containerized with Docker
- Database: PostgreSQL with read replicas for scaling
- Cache: Redis for session storage and API response caching
- File Storage: AWS S3 for project briefs and attachments

- CDN: CloudFront or Cloudflare for static assets
- Load Balancer: Nginx or AWS ELB for traffic distribution
- CI/CD: GitHub Actions for automated testing and deployment
- Monitoring: Sentry (error tracking), DataDog (performance)
- Logging: ELK stack (Elasticsearch, Logstash, Kibana)

13. Future Enhancements (Post-MVP)

- Mobile App: React Native or Flutter for iOS/Android
- Integrations: Slack, Microsoft Teams, Jira, GitHub
- Advanced Analytics: Custom reports, data exports, charts
- Resource Planning: Budget tracking, expense management
- Time Off Management: PTO calendar, availability updates
- Gamification: Badges, leaderboards for task completion
- AI Chatbot: Natural language project queries
- Video Conferencing: Integrated Zoom/Google Meet
- Document Collaboration: Real-time editing like Google Docs
- Multi-language Support: Internationalization (i18n)

14. Conclusion

SkillSync represents a modern approach to project management that leverages AI to optimize team composition, predict project outcomes, and intelligently allocate resources. By automating tedious staffing decisions and providing data-driven insights, the platform empowers organizations to deliver projects more efficiently and with higher success rates. This SRS provides a comprehensive blueprint for building an MVP that can be iteratively enhanced based on user feedback and evolving organizational needs.